Agentes Inteligentes (AIN)

Práctica 1: Sesión 2 Evaluacion en ADK de Google. Trabajo a realizar

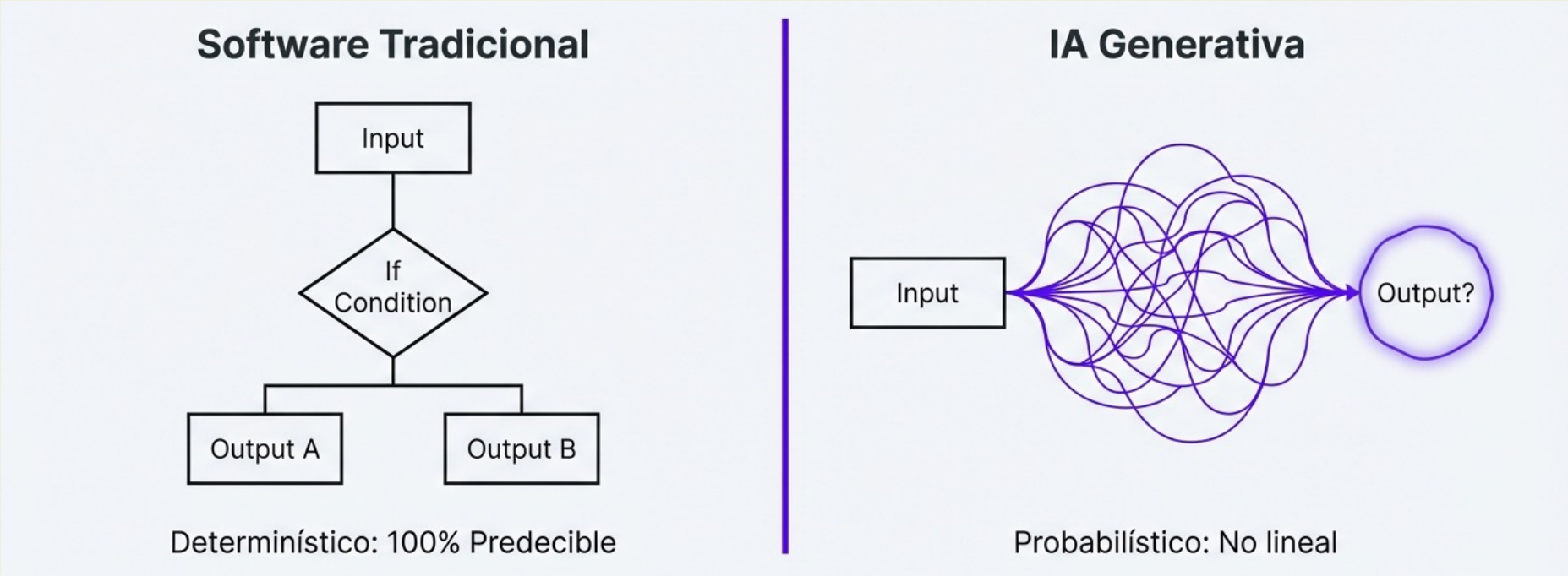
ADK (agent development kit)

Objetivos

- Conocer cómo poder evaluar el comportamiento de los agentes con el ADK de Google
- Plantear el trabajo práctico a realizar

ADK (agent development kit)

Evaluación



- La IA generativa no es determinística.
- No puedes escribir una simple instrucción `if` para verificar la corrección.
- No puedes corregir lo que no puedes medir.

ADK (agent development kit)

Evaluación

La Trayectoria

¿Usó las herramientas correctas en el **orden** correcto?

Ejemplo (JetBrains Mono):
`check_inventory` -> `place_order`

Lógica de Negocio

La Respuesta

¿Es la respuesta final correcta, segura y cortés?

Ejemplo:
Gramática, Tono, Precisión de Datos

Calidad del Lenguaje (QA)

- La evaluación de agentes es más compleja que la evaluación estándar de LLMs.
- No solo calificas el **Ensayo** (respuesta final), sino también **la lógica o las herramientas** que se usaron para llegar a la respuesta.

ADK (agent development kit)

Evaluación

La evaluación en ADK analiza **proceso + respuesta**, no solo el resultado final. Pasos:

Crear Dataset de referencia (Golden Dataset)

- Interacción manual con el agente (IU web)
- Captura de **trayectoria de herramientas**
- Verificación de trazas: Validar lógica correcta (no "alucinación afortunada")

Definición de criterios de éxito

- Ejecución de pruebas desde la interfaz de ADK
- Definición de test_config.json con umbrales de trayectoria y métricas de respuesta:

Realizar pruebas de regresión (CLI - adk eval)

- Ejecución automática contra el dataset de pruebas (creado manualmente si es necesario)
- Reporte de tests pasados/fallidos

Depuración

- Error de trayectoria → ajustar prompts/instrucciones
- Error de respuesta → ajustar métricas o evaluador

Integración CI/CD (no lo vamos a ver)

- Automatización con Pytest

ADK (agent development kit)

Evaluación

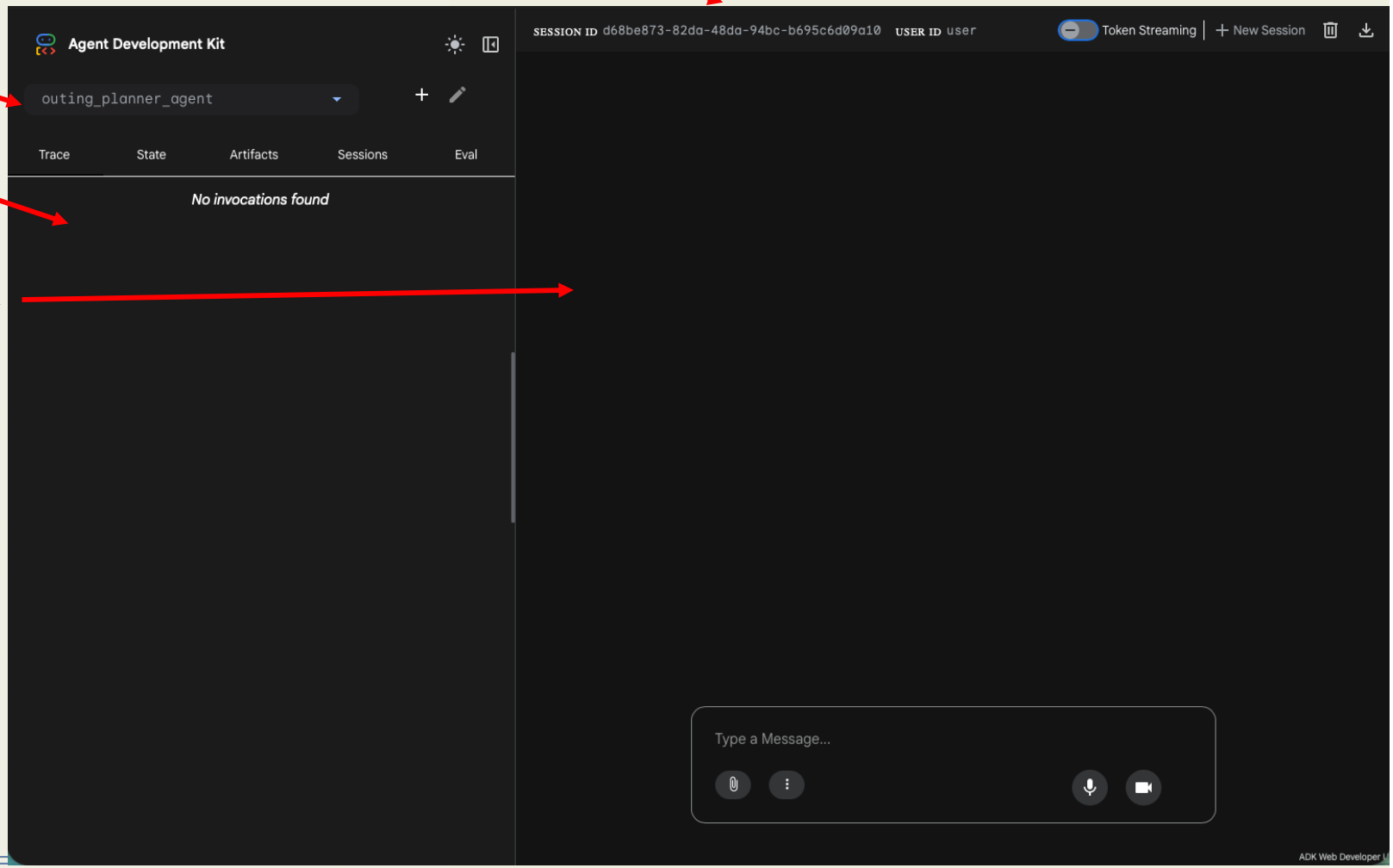
Interfaz Web

Control de sesión

Selector del agente

Gestión del agente

Ejecución y conversación



ADK (agent development kit)

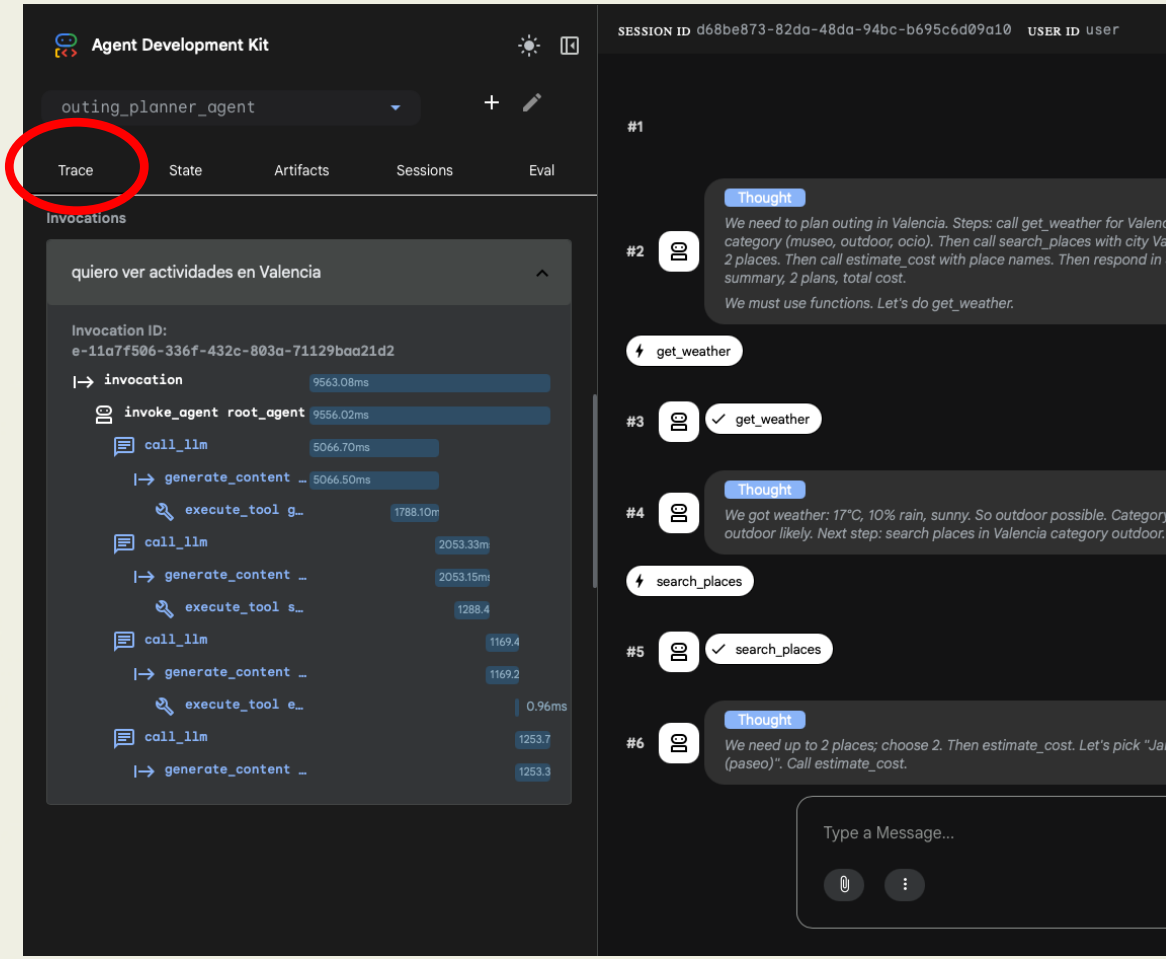
Evaluación

Trace

Muestra la traza completa de ejecución del agente:

- llamadas a herramientas
- decisiones del agente
- pasos de razonamiento
- mensajes generados

Sirve para debugging del flujo del agente.



ADK (agent development kit)

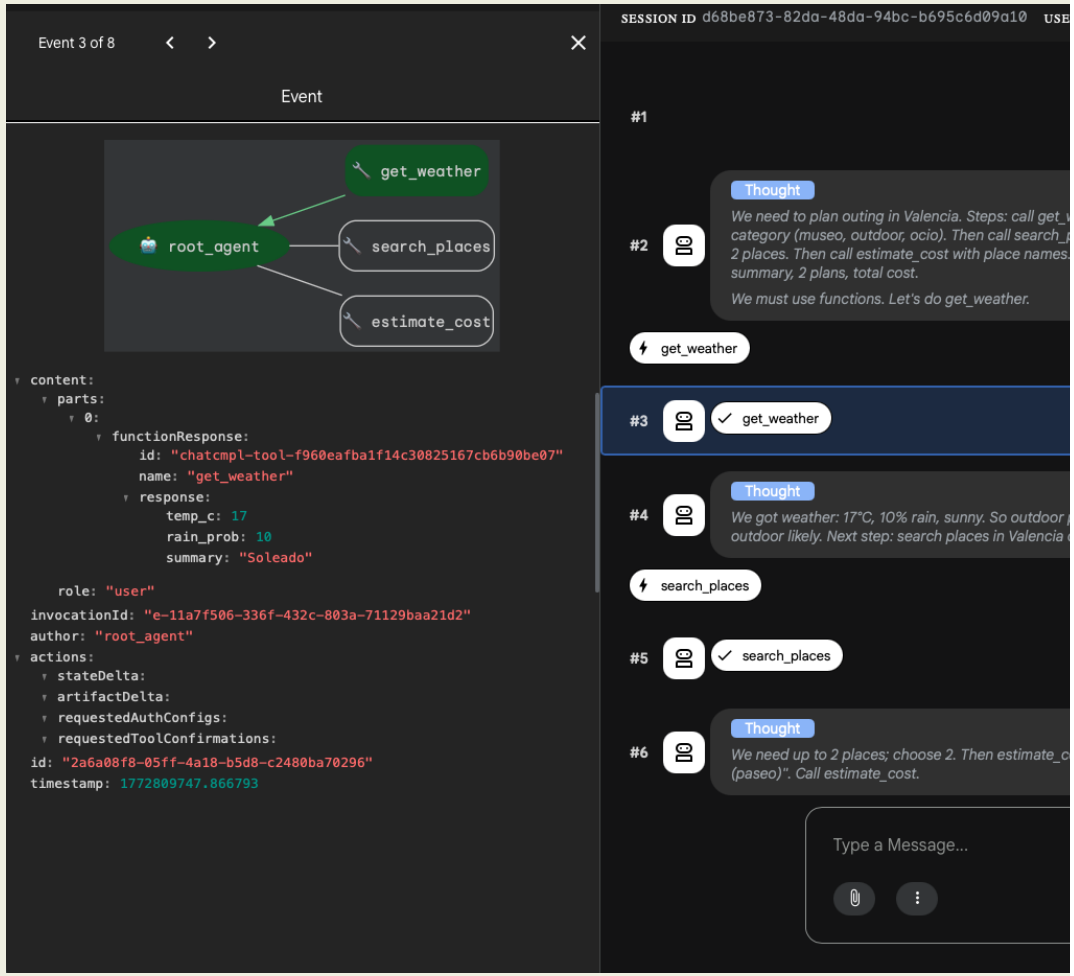
Evaluación

Si seleccionamos un paso de la ejecución en la parte derecha … se detallan los eventos

Cada evento representa una acción del agente durante su ejecución, por ejemplo:

- recepción de un mensaje del usuario
- generación de un pensamiento (reasoning)
- llamada a una herramienta (tool)
- respuesta de una función
- generación de la respuesta final

Esto permite analizar paso a paso cómo razona el agente.



ADK (agent development kit)

Evaluación

Sessions

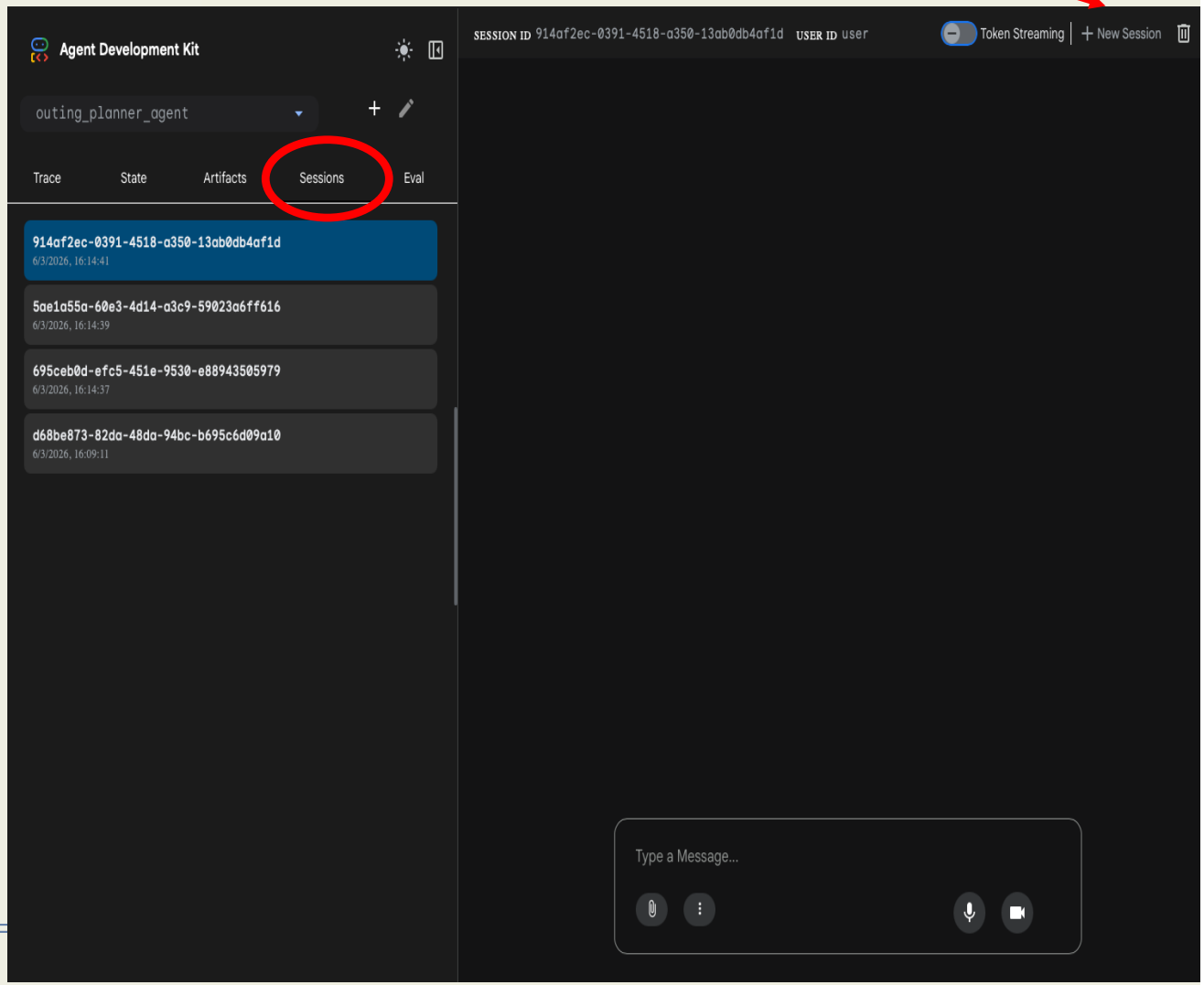
Lista las sesiones de ejecución del agente.

Cada sesión contiene:

- historial de mensajes
- ejecución completa
- contexto del usuario

Permite reproducir ejecuciones anteriores.

Crear nueva sesión



ADK (agent development kit)

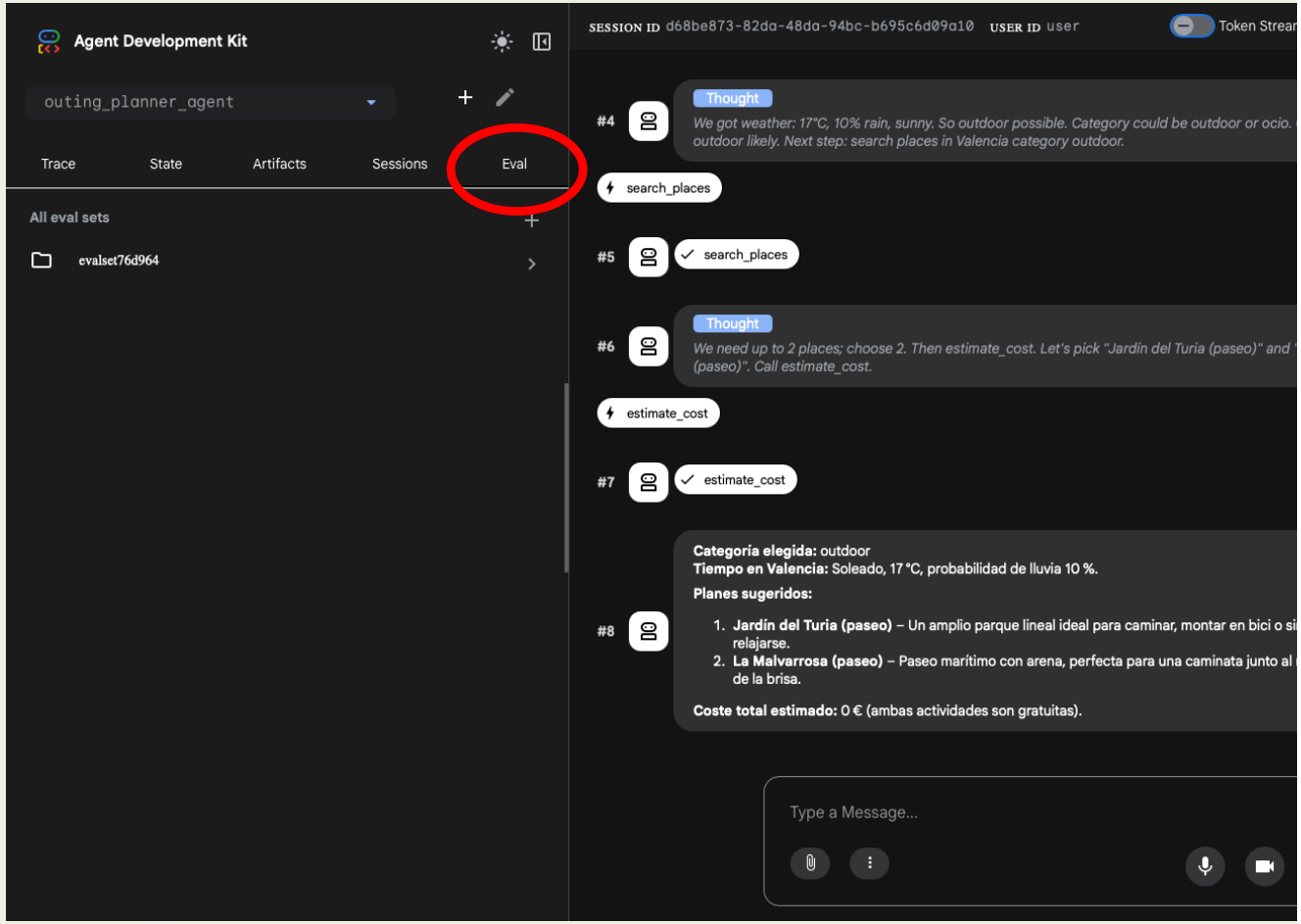
Evaluación

Eval

Se utiliza para **evaluar el comportamiento del agente**.

Normalmente se emplea para:

- pruebas automáticas
- benchmarks
- validación de prompts o herramientas



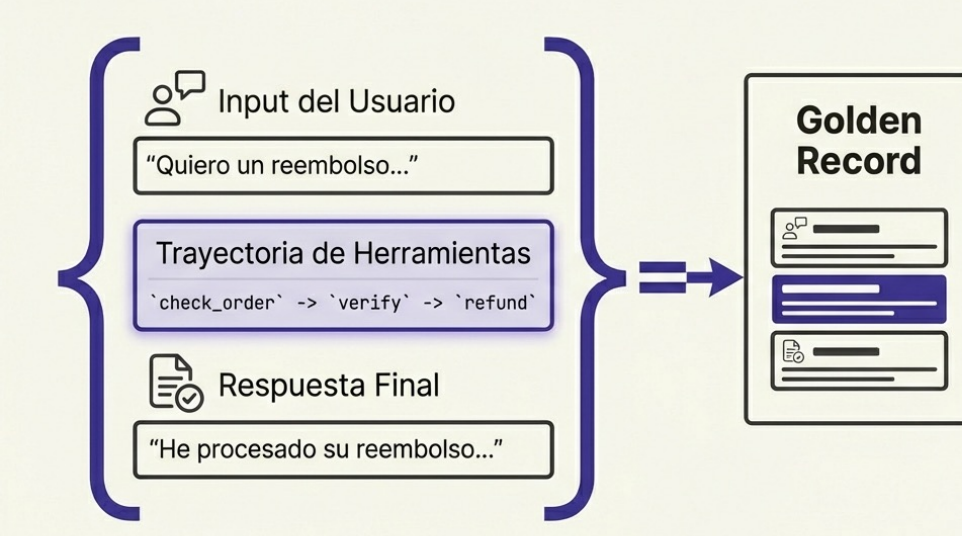
ADK (agent development kit)

Evaluación

Golden Dataset

¿Qué es un conjunto de datos de referencia?
Un conjunto de datos de referencia es una **instantánea de tu agente cuando funciona correctamente**.

No es solo una colección de preguntas y respuestas. Captura el comportamiento completo del sistema:
- **La intención del usuario**
(“Quiero cambiar mi dirección de envío”)
- **La trayectoria interna**
(Secuencia exacta de llamadas a herramientas: get_user_profile → validate_new_address → update_shipping_address → send_email)
- **La respuesta final ideal**
(“Tu dirección ha sido actualizada correctamente. Te hemos enviado un correo de confirmación.”)



ADK (agent development kit)

Evaluación

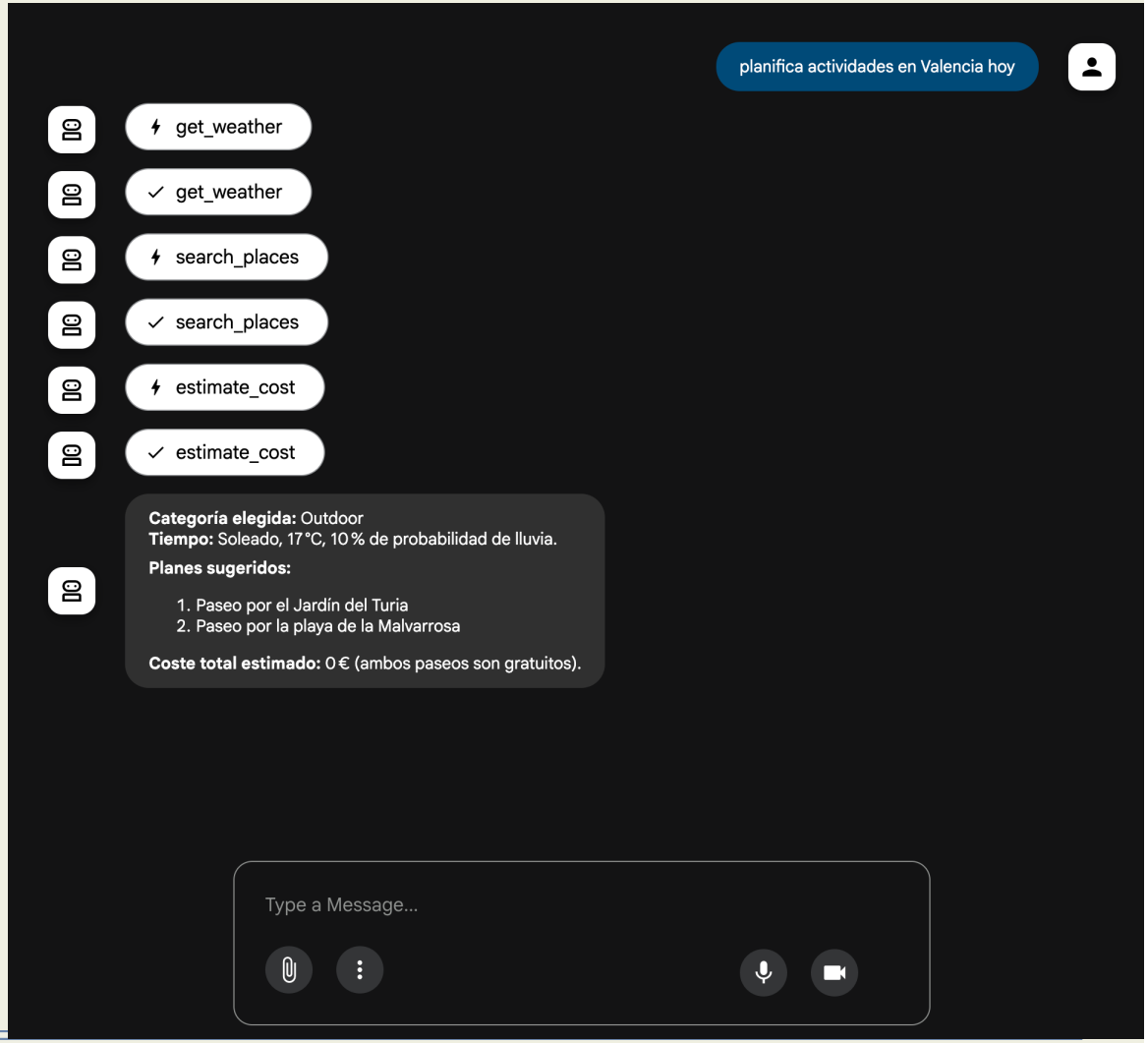
Captura y creación

Generando datos reales con la web UI de ADK

$ adk web

Capturamos el flujo ideal:

- **orden correcto de llamadas a tools**
- **resultado correcto**



ADK (agent development kit)

Evaluación

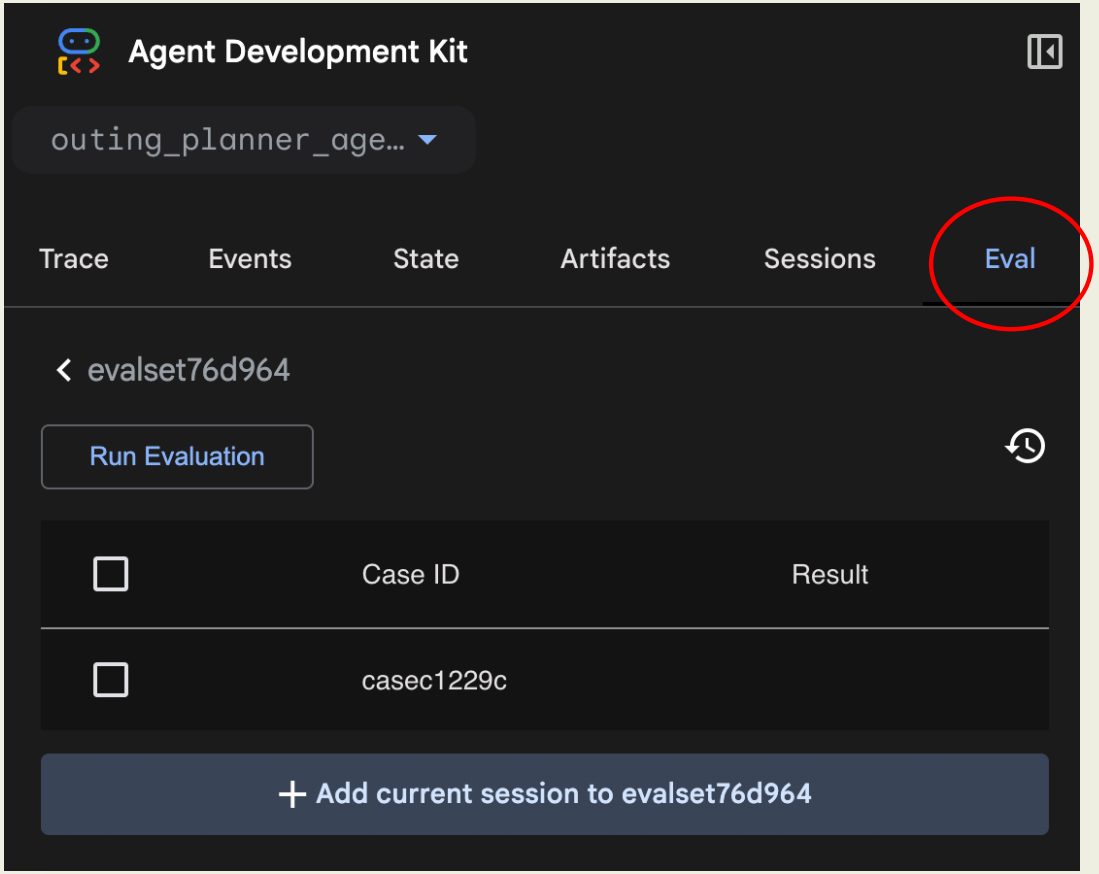
Captura y creación

Generando datos reales con la web UI de ADK

$ adk web

Capturamos el flujo ideal:

- **orden correcto de llamadas a tools**
- **resultado correcto**
- **Se guarda como ejemplo de evaluación**



ADK (agent development kit)

Evaluación

Métricas y configuración

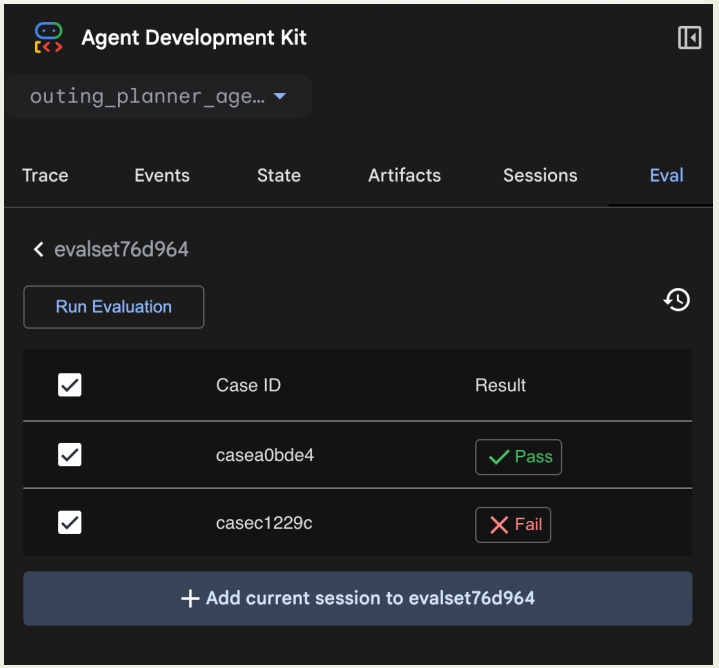
tool_trajectory_avg_score
Mide la precisión en el uso de las herramientas.
Por defecto el criterio es EXACTO.
Objetivo: 0.8 (El agente siguió el proceso correcto al menos en un 80%).

response_match_score
Usa ROUGE-1 (Coincidencia de palabras respecto a un texto base).
Rápido y determinístico. No semántica.

Avanzado: Usar Jueces LLM para evaluación semántica (próxima sesión).

test_config.json

```
{
  "criteria": {
    "tool_trajectory_avg_score": 0.8,
    "response_match_score": 0.5
  }
}
```



ADK (agent development kit)

Evaluación

pip install "google-adk[eval]"

Ejecución por CLI

adk eval

```
$ adk eval agent_name test_file_json

Opcional
--config_file_path=test_config_file.json
--print_detailed_results
```

Ejemplo

```
$ adk eval outing_planner_agent
outing_planner_agent/outing_planner_manual.eval.test.json \
  --config_file_path=outing_planner_agent/test_config.json
```

Ejemplo fichero de pruebas

```
{
  "eval_set_id": "<eval_set_id>",
  "name": "<nombre_del_conjunto>",
  "eval_cases": [
    {
      "eval_id": "<id_del_caso>",
      "session_input": {
        "app_name": "<nombre_app>",
        "user_id": "<id_usuario>",
        "state": {}
      },
      "conversation": [
        {
          "invocation_id": "<id_turno>",
          "user_content": {
            "role": "user",
            "parts": [
              { "text": "<mensaje_usuario>" }
            ]
          },
          "final_response": {
            "role": "model",
            "parts": [
              { "text": "<respuesta_esperada>" }
            ]
          }
        }
      ]
    }
  ]
}
```

Ejemplo fichero de configuración

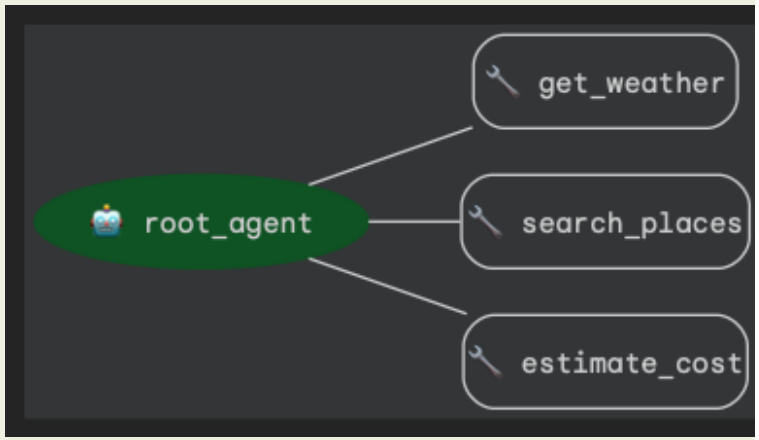
```
{
  "criteria": {
    "tool_trajectory_avg_score": 0.8,
    "response_match_score": 0.3
  }
}
```

ADK Google

Ejemplo

Objetivo del ejemplo

- Agente de búsqueda de actividades en una ciudad
- Uso de datos estáticos (CSV) mediante tools para recomendar planes.
- La idea es medir: Orden de tools, formato y coherencia de respuesta.



Componentes

- Datos: weather.csv (ciudad → temp/rain/summary) y places.csv (ciudad+categoría → lugares).
- Tools: get_weather(city), search_places(city, category), estimate_cost([names]).
- Agente raíz: Agente con instrucciones paso a paso

Qué evaluar en ADK

- Cobertura de tools: ¿siempre llama a get_weather → search_places → estimate_cost?
- Respuesta: coincide en un % con la respuesta esperada

ADK Google

Ejemplo

Flujo esperado (trazable en logs)

1) Prompt del usuario (ciudad a visitar)

2) Tool: get_weather(city)

3) Decide categoría (museo | outdoor | ocio)

4) Tool: search_places(city, category)

5) Selecciona hasta 2 lugares

6) Tool: estimate_cost([names])

7) Respuesta final (en español, con coste total)

```
# Descargarse y descomprimir “ejemplo_eval.zip”

# Instalar el complemento
pip install "google-adk[eval]"
```

En el directorio del ejemplo ejecutar:

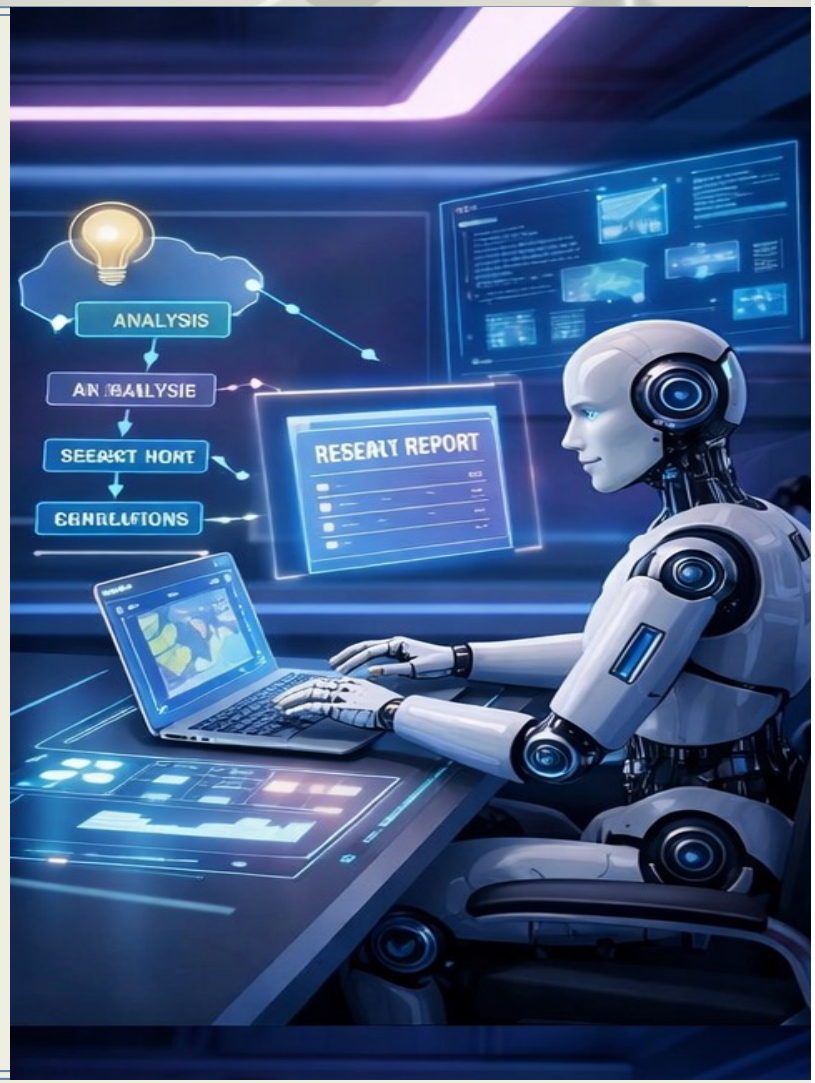
```
(adk)$ adk eval agent agent/manual.eval.test.json
--config_file_path=agent/test_config.json
```

Resultado (algo similar):

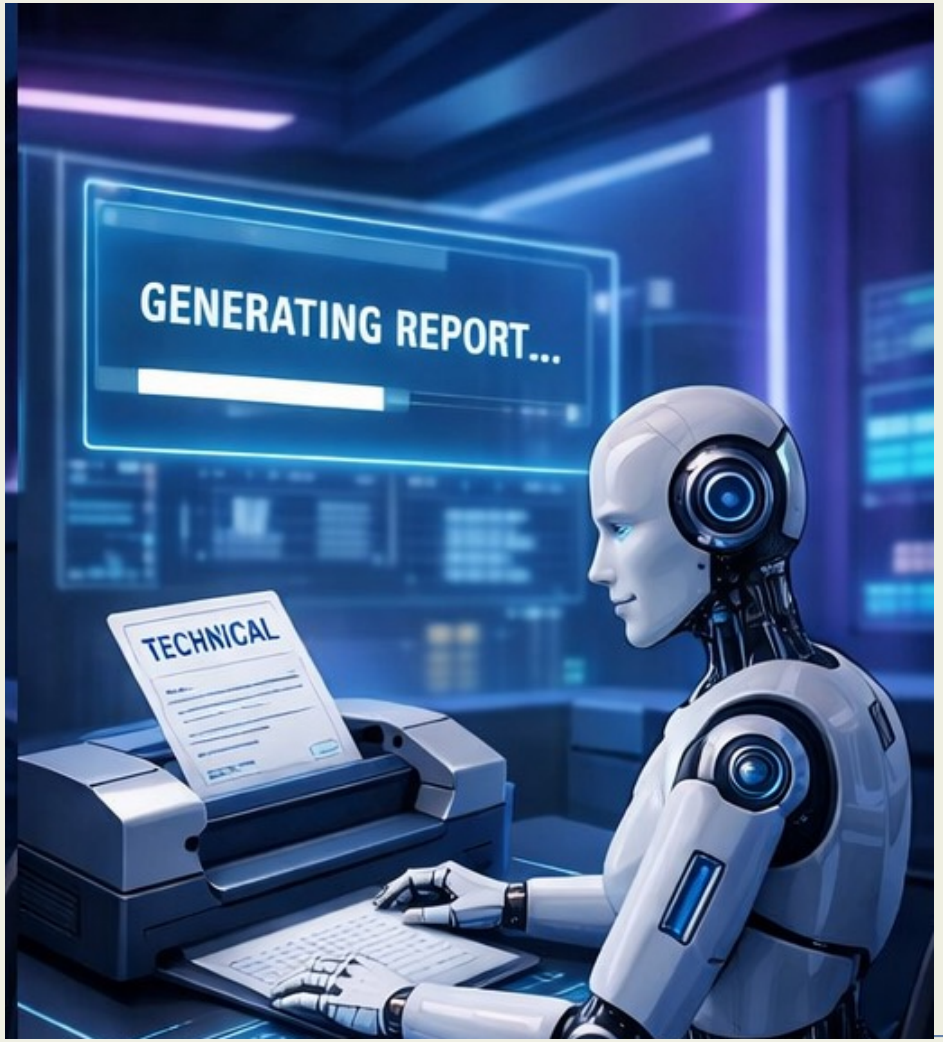
```
**********************************************************************
Eval Run Summary
outing_planner_manual:
  Tests passed: 4
  Tests failed: 1
```

Práctica: Agente generador de documentos

- Práctica en grupo (2 personas)
- El objetivo de esta práctica es diseñar e implementar un **agente** que razone, genere y estructure un documento/informe sobre un tema determinado, que se materialice en un **documento final generado automáticamente por el propio sistema**.
- La práctica está orientada a aplicar los conceptos vistos en ADK relacionados con:
  - Creación de un **agente**
  - Uso de **tools**, **callbacks**, control del contexto
  - Generar **pruebas** para evaluación
- Fecha entrega: **21 o 22 de abril** (Tarea en poliformat)



Práctica: Agente generador de documentos



- El sistema debe generar automáticamente un **documento final** que incluya:
  - Un titulo
  - Introducción
  - Desarrollo estructurado del tema,
  - Conclusiones
  - Bibliografía.

  Y un json con la estructura para facilitar la evaluación automática

- **Entregables**
  - **Código fuente** del sistema, debidamente estructurado y documentado.
  - **Informe técnico** (máximo 2 páginas) que describa la arquitectura del sistema
  - **Documento de ejemplo** generado automáticamente por el agente
  - **JSON generado** de ese mismo ejemplo

Práctica: Agente generador de documentos

- Estructura json de salida

```
object
 |
 ├── title: string
 |
 ├── sections: array<object>
 |     ├── name: string
 |     └── word_count: integer
 |
 ├── total_words: integer
 ├── num_sections: integer
 ├── num_references: integer
 └── pdf_path: string
```

Práctica: Agente generador de documentos

Ejemplo json de salida


```
{
  "title": "Inteligencia Artificial Agéntica",
  "sections": [
    {
      "name": "Introducción",  "word_count": 162
    },
    {
      "name": ”Estado del arte",  "word_count": 157
    },
    {
      "name": ”Desarrollo”,  "word_count": 176
    },
    {
      "name": ”Ejemplos",  "word_count": 172
    },
    {
      "name": "Conclusiones",  "word_count": 151
    }
  ],
  "total_words": 818,
  "num_sections": 5,
  "num_references": 3,
  "pdf_path": "output/informe.pdf"
}
```

Práctica: Agente generador de documentos

2 opciones: 1. El agente genera sobre cualquier cosa (en general) o 2.El agente escribe sobre un tema en específico (un tema concreto)

El tema del documento puede elegirse libremente dentro de los propuestos o por vosotros.

- Ejemplos:
  - Aplicación de modelos de emociones a agentes
  - Physical Agents – MAS en robótica
  - Negociación automática
  - MAS en videojuegos
  - Aprendizaje Federado basado en MAS
  - Agentes y eHealth
  - Agentes y Smart Cities
  - Agentes LLM